

INTERNAL SHARE

project-brain

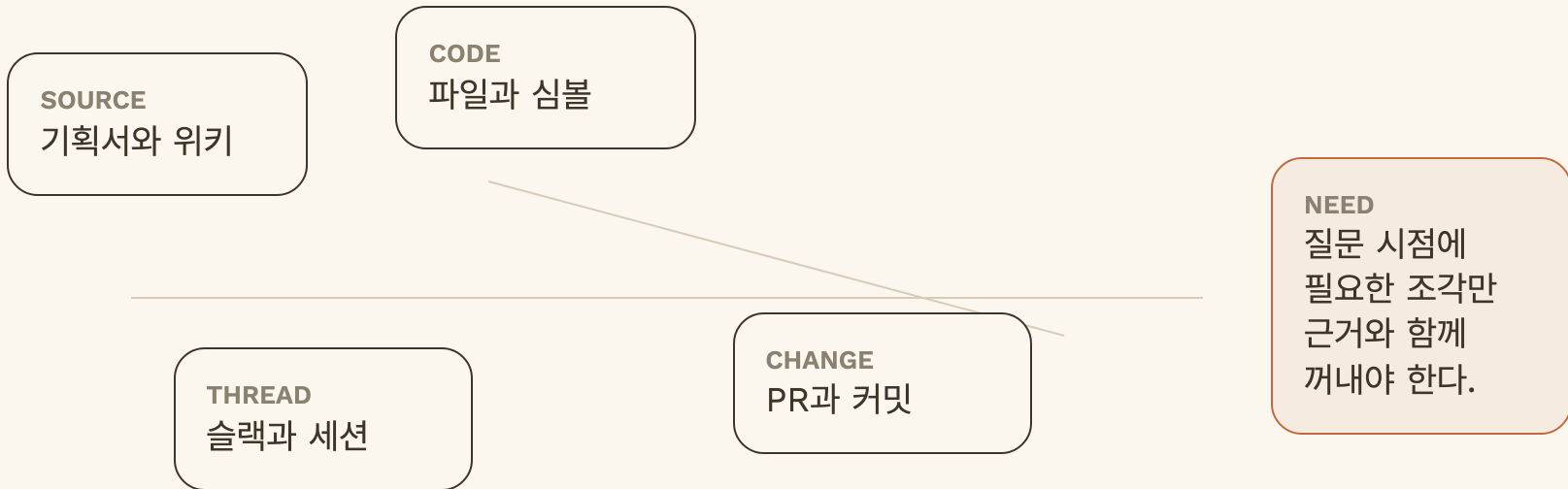
프로젝트 기억을 구조화하고
다시 꺼내 쓰는 엔진



WHY

프로젝트 지식은 흩어지고 사람은 쉽게 잊는다

코드, 기획서, 대화, PR, 결정 기록이 서로 다른 위치에 남는다.



사람이 직접 뒤지는 도구가 아니라, 에이전트가 호출하는 지식 엔진

질문은 사람에게서 오고, 검색과 조립은 에이전트가 맡는다.

1

USER

질문하거나 작업을
맡긴다

2

AGENT

필요한 지식을
판단한다

3

BRAIN

근거와 검수 상태를
회수한다

4

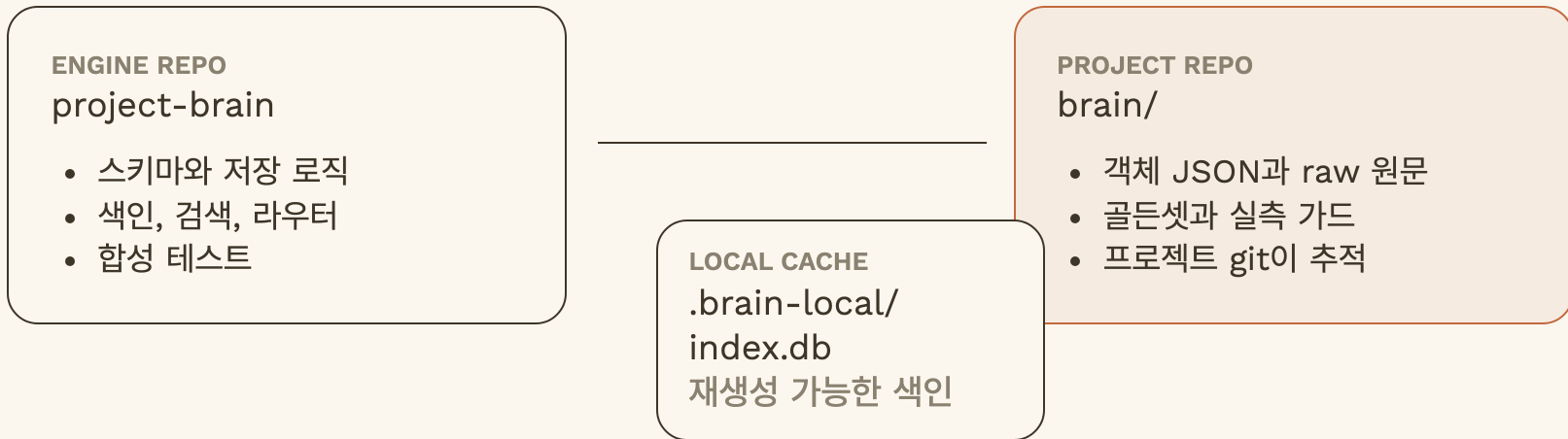
ANSWER

근거 있는 답을
만든다

STORAGE MODEL

엔진과 데이터는 분리된다

엔진은 범용이고, 실제 코퍼스는 각 프로젝트 저장소의 `brain/`이 가진다.



ENGINE DESIGN

원본에서 답까지 얇은 파이프라인

엔진은 판단을 대신하지 않고, 저장·색인·검색·상태 신호를 맡는다.



OBJECT KINDS

타입은 분류표가 아니라 역할의 경계다

같은 “지식”이라도 원본, 용어, 매핑, 코드, 결정, 교훈은 서로 다른 검증 규칙을 갖는다.

SOURCE

EvidenceManifest

EvidenceRef

원본과 원본 안의 특정
근거 위치

DOMAIN

DomainContext

GlossaryTerm

DomainMapping

용어, 경계, 의미 연결

IMPLEMENTATION

CodeLocator

DecisionRecord

어디에 있고 왜
정했는가

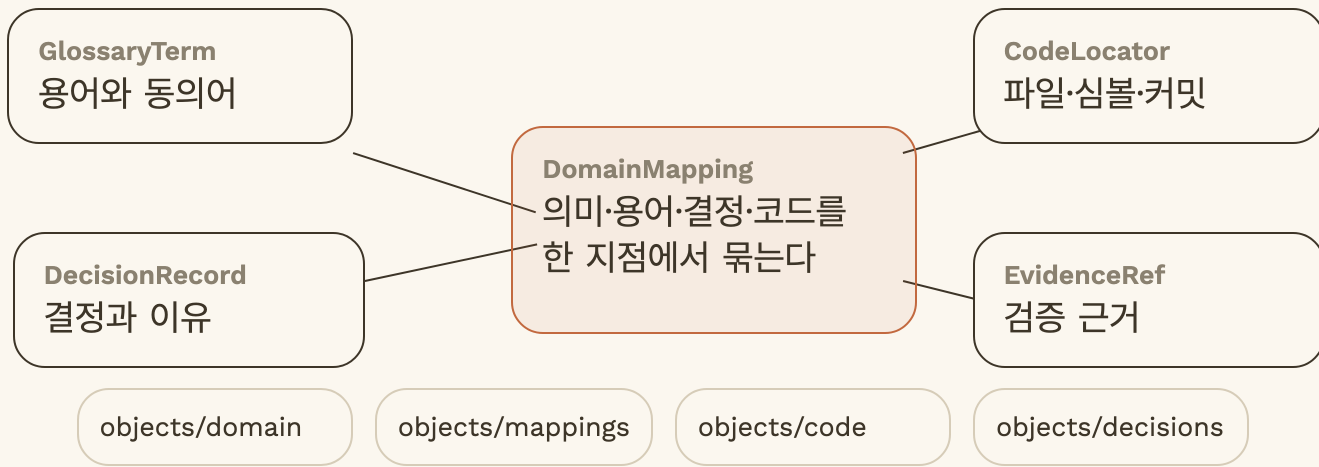
REUSE

Insight

ContextProjection

여러 객체를
가로지르는 교훈과
재사용 브리핑

객체 id 필드가 그래프를 만든다



RECALL

불러올 때는 텍스트와 연결을 같이 쓴다

surface.py가 타입별 검색 표면을 만들고, 검색기는 그 위에서 여러 신호를 합친다.

QUERY

질문

SURFACE

kind별 텍스트
표면

BM25

토큰 기반 키워드

VECTOR

의미 임베딩

GRAPH

1-hop 상호지지

reviewed

candidate

raw

Insight

projection

OPERATING LOOP

근거로 상태를 정해 쌓는다

많이 저장하는 것보다 근거와 상태를 유지하는 운영 흐름이 더 중요하다.



AGENT USE

install은 프로젝트에 config와 brain 스킬 4종을 심고, 에이전트의 절차를 고정한다.

스킬이 에이전트의 사용법을 고정한다

PROJECT INSTALL

project-brain install
.project-brain.json
.project-brain-
manifest.json

brain-query

먼저 search / show
없으면 없다고 말한다

brain-ingest

완료된 기능을
객체 그래프로 적재

brain-session-ingest

개발 중 결정과
세션 지식 갱신

brain-audit

lint, isolated,
stale-check 점검

.agents/skills/
<project>-brain-*

검색·적재·점검의
절차를 문서로 고정

사용자 수정 파일은
manifest로 보존

스킬은 에이전트가 언제 찾고, 언제 저장하고, 언제 점검할지 정한다

TEAM USE

핵심은 많이 쌓기가 아니라 다시 쓸 수 있게 쌓기

근거, 검수 상태, 운영 루틴이 유지될 때
프로젝트 기억이 팀 자산이 된다.

